

Can LLMs Infer Political Parties’ Strategic Behavior? Evidence from Spatial Voting Models

Yulu Pi
yulu.pi@warwick.ac.uk
University of Duisburg-Essen
Germany
University of Warwick
United Kingdom

Aojie Ju
aojieju2@illinois.edu
University of Illinois Urbana-Champaign
United States

Abstract

Before LLMs can credibly serve as political simulation agents to approximate political actors, they must first demonstrate an accurate understanding of the strategic behaviors they are expected to emulate. We introduce a validation task that tests this prerequisite: given only time-series data on party positions and vote shares from a Laver (2005) multiparty competition model, LLMs must infer which of five adaptive strategies governs each party’s movement, without access to strategy labels. Across six model configurations spanning three families, overall accuracy ranges from 9.4% to 39%, with substantial variation by strategy type and model choice. These results reveal that current LLMs lack reliable comprehension of the competitive dynamics they would need to reproduce, raising a foundational concern for their ready deployment as simulation agents for political dynamics.

Keywords

LLM validation, agent-based simulation, spatial voting, strategy inference, political competition

1 Introduction

Large language models (LLMs) have demonstrated some capabilities in simulating human behavior and are increasingly proposed as agents in simulations of social and political dynamics [3, 4]. However, a fundamental prerequisite precedes their deployment in such settings: before we rely on LLMs to generate realistic social or political behavior as simulated agents, we should establish whether they can infer relevant behaviors from evidence – an ability that is fundamental to realistic social and political simulation. For instance, if a model cannot recognize distinct party strategies from observed trajectories and electoral outcomes in a controlled environment, it would be risky to trust it to produce those strategies when acting as a simulated agent. Our concern is consistent with recent associated work, as many have highlighted validation as a key challenge for LLM-based simulations [1, 2].

To operationalize this prerequisite, we designed an exploratory strategy-inference task. 6 LLMs are provided with time-series data on party positions and vote counts from 50 simulations based on Laver’s (2005) spatial voting model, in which parties follow five distinct adaptive strategies. We then evaluate whether models can correctly identify the strategy employed by each party using only the observed patterns of movement and vote share. Our results

reveal substantial variation across models, with overall accuracy ranging from 9.4% to 39%. These findings suggest that current LLMs demonstrate a partial but incomplete understanding of political strategic behavior, with some inconclusive evidence that understanding might improve with larger model size. However, the observed variability and systematic low accuracy indicate that these models are not yet reliable enough to be treated as fully trustworthy agents in political simulations, highlighting the need for careful validation with domain experts before their deployment in political science research or policy contexts.

2 Spatial Voting Models

From the scope of political behavior, spatial models of party competition have long served as a foundational framework for understanding electoral dynamics. In the classic work of Downs [6], parties are modeled as vote-seeking competitors who choose policy positions in an ideological space to maximize voter support, while voters select the party closest to their own positions. Laver [11] extended this framework to study dynamic competition with adaptive strategies, where parties adjust positions based on different behavioral rules rather than assuming perfect rationality or equilibrium-seeking. In this model, parties occupy positions in a two-dimensional space representing economic policy and social policy. Voters vote for the closest party by Euclidean distance, and parties update positions according to one of five strategies:

- **Sticker:** Never moves — stays fixed in place
- **Predator:** Moves toward the position of the largest party
- **Aggregator:** Moves toward the average position of its current supporters
- **Hunter:** Repeats moves that gained votes; otherwise changes direction randomly
- **Random:** Moves in a random direction each step

In this work, we leverage Laver’s (2005) spatial voting framework as a foundation for evaluating the ability of LLMs to infer and reason about political strategy, providing a concrete benchmark for simulation readiness in social and political contexts. LLM agents are increasingly proposed as agents of large-scale social simulations [3, 4], but recent survey analysis emphasizes that validation and reliability rather than narrative plausibility serve as the primary weakness for policy-facing deployments [7–9]. These patterns can recur in simulations of international relations, regulatory policymaking, and social movements, where actors must navigate complex strategic landscapes. Critically, if LLMs cannot identify and apply these fundamental strategic patterns in a controlled electoral

This paper was prepared for PoliSim@CHI 2026: LLM Agent Simulation for Policy, a workshop at CHI 2026 (CHI Conference on Human Factors in Computing Systems), April 16, 2026, Barcelona, Spain.

setting, they are unlikely to demonstrate the reasoning needed for authentic simulation across other policy domains. By using spatial voting as a structured testbed, we can assess whether LLMs possess the strategic reasoning capabilities required for reliable deployment as simulation agents.

3 Experimental Design and Results

Simulation Setup. We generated 50 independent simulations implementing Laver’s (2005) spatial voting model. Each simulation ran for 100 time steps with: 3–7 parties per simulation (uniformly sampled); 500–1,000 voters per simulation (uniformly sampled); initial positions randomly sampled from $[0, 1]^2$ for all agents; and constrained randomization ensuring each of the 5 strategies appeared at least 10 times across all simulations. This yielded 254 total party instances with the following distribution: Aggregator (45), Hunter (52), Predator (58), Random (53), Sticker (46). At each time step, we recorded party position (x, y), strategy label, and vote count. Figure 1a displays initial and final party positions in two-dimensional ideological space for a representative simulation, while Figure 1b presents the corresponding vote count dynamics over 100 timesteps for the same simulation, demonstrating how different strategic behaviors translate into electoral outcomes.

Inference Task. For each simulation, we provided models with time-series data containing party identifiers, positions (x, y) at each time step, and vote counts at each time step. Critically, we withheld strategy labels and asked models to predict which of the five strategies each party was following. To manage input length, we provided a 30-step summary consisting of the first 10, middle 10, and last 10 timesteps for each party (out of 100 total). The prompt included: (i) definitions of the five strategies, (ii) a description of the 2D ideological space and voting rule, (iii) tabular time-series data, and (iv) instructions to output a JSON mapping from party to predicted strategy. See Appendix A for the complete prompt template we used.

Models. We tested six LLM configurations spanning three model families: (1) GPT-4o, (2) MiniMax-M2.1, (3) Qwen2.5-7B-Instruct, (4) Qwen3-0.6B, (5) Qwen3-1.7B, and (6) Qwen3-30B-A3B. The temperature was fixed at 0.2 across all runs to ensure consistency. This selection enables three key comparisons. First, the Qwen family allows systematic testing of whether strategic understanding scales with model size, a critical question for practitioners choosing between efficiency and capability. Second, including both proprietary frontier models (GPT-4o, MiniMax-M2.1) and open-weight alternatives (Qwen) addresses practical deployment considerations, as open-weight models offer reproducibility and cost advantages increasingly important for simulation research. Third, spanning different model families (OpenAI’s GPT, Alibaba’s Qwen, MiniMax’s architecture) tests whether strategic inference depends on specific training approaches or generalizes across architectures, informing whether this capability reflects fundamental model understanding versus family-specific artifacts. As a reference point, a uniform random predictor over five strategies yields 20% expected accuracy.

Model	Correct	Total	Accuracy
GPT-4o	99	254	39.0%
MiniMax-M2.1	64	254	25.2%
Qwen2.5-7B-Instruct	45	254	17.7%
Qwen3-0.6B	30	254	11.8%
Qwen3-1.7B	26	254	10.2%
Qwen3-30B-A3B	24	254	9.4%

Table 1: Overall strategy identification accuracy across six LLM configurations (chance baseline = 20%).

Strategy	GPT-4o	MiniMax	Qwen2.5	Qwen3-0.6	Qwen3-1.7	Qwen3-30B
Sticker	100.0	71.7	39.1	41.3	21.7	26.1
Hunter	51.9	28.8	11.5	1.9	5.8	11.5
Aggregator	26.7	8.9	26.7	17.8	4.4	6.7
Random	22.6	17.0	5.7	3.8	7.5	0.0
Predator	3.4	5.2	10.3	0.0	12.1	5.2

Table 2: Per-strategy accuracy (%) across models. Predator is consistently the hardest strategy; Sticker is the easiest.

3.1 Overall Performance

Table 1 and Figure 2 reports overall accuracy across models. Results revealed substantial variation in inference capability, with accuracy ranging from 9.4% - 39% across models. GPT-4o achieved the best performance (39.0%), outperforming the second-best model (MiniMax-M2.1, 25.2%). All Qwen variants performed below the random baseline.

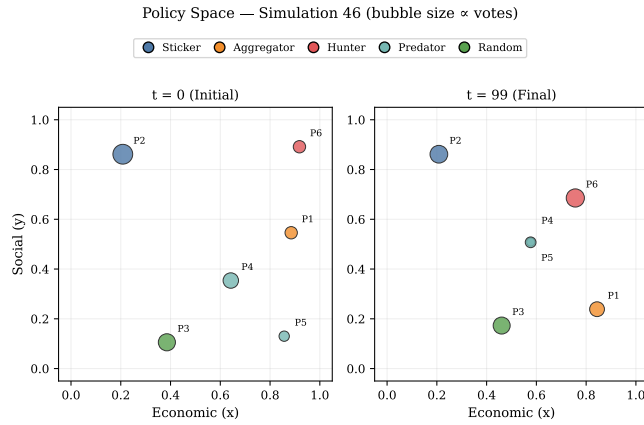
3.2 Per-Strategy Accuracy and Error Patterns

Table 2 and Figure 3 reports per-strategy accuracy. Across all models, Sticker is the easiest strategy to identify (note that GPT-4o reaches 100%), consistent with the fact that sticker parties have near-zero position variance. In contrast, Predator is consistently the hardest strategy, with all models performing poorly (best observed: 12.1%). Models also show substantial confusion between Hunter and Random, likely because both may present non-smooth trajectories unless vote-conditioned repetition is detected.

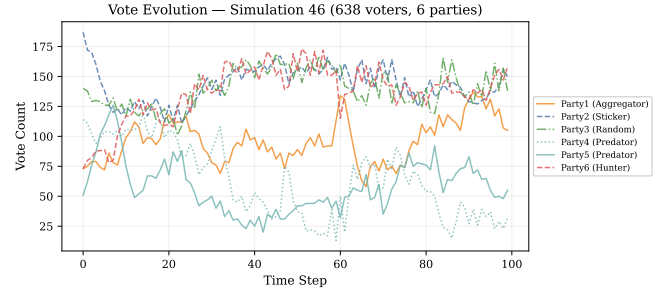
4 Discussion

Sticker detection as a validity floor. Sticker parties do not move, making them easily separable based on position variance. All models achieve their highest per-strategy accuracy on Sticker, with GPT-4o reaching 100% correct identification. This establishes a minimum competency threshold: models failing to detect non-movement lack the observational capacity for simulation work. For practitioners, Sticker accuracy serves a diagnostic role. If a model cannot reliably identify fixed positions, it should not be deployed as an agent in dynamic simulations.

Predator requires multi-party relational reasoning. Correctly identifying Predator requires the model to (i) infer which party is currently largest by votes, (ii) compute whether a party’s movement vector is directed toward that largest party, and (iii) handle target switching when the largest party changes over time. This multi-step, cross-party inference appears challenging for current models.



(a) Policy space at $t=0$ (left) and $t=99$ (right). Bubble size indicates vote count.



(b) Vote count evolution over 100 time steps.

Figure 1: Simulation 46 (638 voters, 6 parties): (a) Party positions in policy space, (b) corresponding vote dynamics.

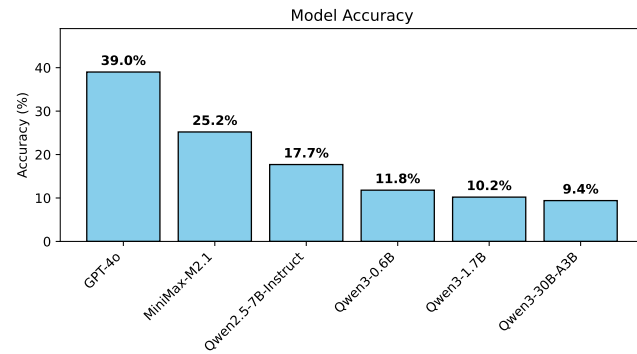


Figure 2: Overall strategy inference accuracy by model.

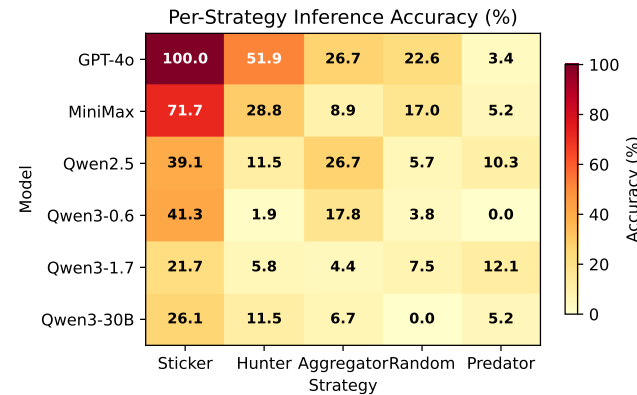


Figure 3: Per-strategy accuracy (%) across models. Sticker (zero positional variance) is consistently the easiest to identify; Predator is the hardest across all models.

Hunter vs Random confusion. Both strategies can exhibit irregular trajectories. Distinguishing Hunter requires tracking whether

direction repetition is conditioned on vote gains, which may be difficult under subsampling and limited context.

Scale alone does not guarantee performance. Despite being the largest open-source configuration tested, Qwen3-30B-A3B performs worst overall, suggesting that architecture/training and structured-reasoning robustness may matter more than parameter count for this task. Practitioners cannot assume larger models are automatically more simulation-ready than smaller model. Task-specific validation is essential.

Implications for simulation practice. Our task tests a necessary precondition: to simulate political competition, an LLM agent must understand how different strategies manifest in observable outcomes. The results show that current models largely fail this precondition. These findings establish that current LLMs require domain expert validation before deployment as simulation agents. Practitioners should: (1) test candidates on domain-specific strategy inference tasks rather than relying on general accuracy or parameter counts, (2) examine error patterns to identify which behaviors the model misunderstands, and (3) establish minimum competency thresholds (e.g., $> 95\%$ on simple strategies like Sticker) before trusting models with complex multi-agent dynamics. The substantial performance variation across models (9.4-39% accuracy) suggests this validation step can meaningfully improve simulation quality by steering practitioners toward models with stronger strategic comprehension.

5 Limitations and Future Work

As the first exploratory investigation of LLM strategy inference from quantitative political simulation data, our study prioritizes establishing baseline feasibility over conducting an exhaustive evaluation. Several design choices and constraints warrant discussion and suggest directions for future work. First, our dataset includes 254 party instances across 50 simulations, a scale sufficient for detecting large-magnitude effects in exploratory studies. While this sample size limits statistical power for fine-grained subgroup analyses, it is adequate to establish our core finding: even state-of-the-art

models achieve only moderate accuracy (39%), highlighting fundamental limitations in inference. Second, to satisfy input length constraints, models observe only 30 of the 100 timesteps. This restriction may obscure vote-conditioned repetition patterns needed to identify Hunter behavior, as well as the dynamic “chase-the-leader” signals characteristic of Predator strategies. Third, we fixed both the prompt format and temperature (0.2) to ensure comparability across models. However, model performance may be sensitive to these choices. Systematic prompt engineering, such as incorporating chain-of-thought reasoning or explicit strategy comparisons, represents a promising avenue for improving inference accuracy in future work.

6 Conclusion

Our results demonstrate that even state-of-art LLMs achieve only modest accuracy in inferring political party strategies from quantitative simulation data, revealing fundamental limitations in their understanding of competitive dynamics. The substantial performance variation across models indicates that careful, task-specific validation by domain experts is essential before deploying LLMs in political simulations. For the growing community building LLM-based agent simulations, our work provides both a validation methodology and an empirical baseline, demonstrating that strategy inference tasks offer practitioners a concrete mechanism for assessing whether candidate models possess the understanding their simulations require.

References

- [1] Guo, T., Chen, X., Wang, Y., Chang, R., Pei, S., Chawla, N. V., Wiest, O., & Zhang, X. 2024. Large Language Model based Multi-Agents: A Survey of Progress and Challenges. *arXiv preprint arXiv:2402.01680*. <https://arxiv.org/abs/2402.01680>.
- [2] Larooij, M. & Törnberg, P. 2025. Validation is the central challenge for generative social simulation: a critical review of LLMs in agent-based modeling. *Artificial Intelligence Review*, 59(1):15. <https://doi.org/10.1007/s10462-025-11412-6>.
- [3] Park, J. S., O'Brien, J. C., Cai, C. J., Morris, M. R., Liang, P., & Bernstein, M. S. 2023. Generative Agents: Interactive Simulacra of Human Behavior. *arXiv preprint arXiv:2304.03442*. <https://arxiv.org/abs/2304.03442>.
- [4] Argyle, L. P., Busby, E. C., Fulda, N., Gubler, J. R., Rytting, C., & Wingate, D. 2023. Out of One, Many: Using Language Models to Simulate Human Samples. *Political Analysis*, 31(3):337–351. <https://doi.org/10.1017/pan.2023.2>.
- [5] Cao, L., et al. 2024. Can Large Language Model Agents Simulate Human Trust Behaviors? *arXiv preprint arXiv:2402.04559*.
- [6] Downs, A. 1957. *An Economic Theory of Democracy*. Harper & Row.
- [7] Gao, C., Lan, X., Li, N., Yuan, Y., Ding, J., Zhou, Z., Xu, F., and Li, Y. 2024. Large language models empowered agent-based modeling and simulation: a survey and perspectives. *Humanities and Social Sciences Communications* 11(1). DOI: 10.1057/s41599-024-03611-3.
- [8] Huang, Y., Yuan, Z., Zhou, Y., Guo, K., Wang, X., Zhuang, H., Sun, W., Sun, L., Wang, J., Ye, Y., and Zhang, X. 2024. Social Science Meets LLMs: How Reliable Are Large Language Models in Social Simulations? *arXiv preprint arXiv:2410.23426*.
- [9] Kang, J., Han, P., Zhang, T., and Gong, L. 2025. PolicySimEval: A Benchmark for Evaluating Policy Outcomes through Agent-Based Simulation. *arXiv preprint arXiv:2502.07853*.
- [10] Larooij, D. and Törnberg, P. 2025. Can LLM Agents Be Used to Simulate Policy Outcomes? Evaluating the Role of Validation in LLM-Driven Simulations. *arXiv preprint arXiv:2501.07834*.
- [11] Laver, M. 2005. Policy and the Dynamics of Political Competition. *American Political Science Review*, 99(2):263–281.
- [12] McKelvey, R. D. 1976. Intransitivities in Multidimensional Voting Models and Some Implications for Agenda Control. *Journal of Economic Theory* 12(3): 472–482.
- [13] Stokes, D. 1963. Spatial Models of Party Competition. *American Political Science Review*, 57(2):368–377.

A Prompt Template

Below is the prompt template used to query LLMs for strategy inference. The {data_str} placeholder is replaced with the actual simulation data in CSV format (time, party, votes, x, y).

Prompt Template

You are analyzing a political party voting simulation based on Laver (2005).

Parties use one of these strategies:

- Sticker: Never changes position
- Predator: Moves toward the position of the largest party
- Aggregator: Moves toward the center of its current supporters
- Hunter: Repeats moves that increased votes, otherwise moves randomly
- Random: Moves randomly in policy space

Each party has an (x, y) position in a 2D ideological space (values 0-1):

- X-axis: Economic policy (0=Left, 1=Right)
- Y-axis: Social policy (0=Traditional, 1=Progressive)

Voters vote for the closest party (Euclidean distance).

Total number of voters in this simulation: {voter_count if voter_count is not None else 'unknown'}.

Parties move in steps of approximately 0.05 units per time step.

Below is a subset of the simulation data. To reduce size, only three windows of consecutive time steps are shown: the first 10 steps (early behavior), the middle 10 steps (mid-simulation behavior), and the last 10 steps (late behavior). Steps not shown follow the same pattern.

{data_str}

For each party, predict which strategy it is using.

Reply ONLY with a JSON object mapping party names to strategies.

Use exactly these strategy names: Sticker, Predator, Aggregator, Hunter, Random.

Example format: {"Party1": "Sticker", "Party2": "Predator", "Party3": "Hunter"}

Respond with the JSON object for these parties: {'', '.join(party_names)}